

Agentic-Analyst

JGH Intelligence Engine — Enterprise AI SQL Platform Summary

Author / Lead Engineer: Sneha Nayak | Model: Qwen2.5-Coder:7b (Ollama) | Date: September 02, 2026

1. Executive Summary

The **Agentic-Analyst (JGH Intelligence Engine)** is an enterprise-grade, read-only AI SQL agent and conversational Business Intelligence (BI) platform. It allows non-technical business users, analysts, and administrators to query complex relational databases (such as retail distributor-retailer hierarchies, wallet transactions, SKU inventories, and geographical regional data) using natural language English.

Unlike generic text-to-SQL wrappers, the platform incorporates a **multi-stage agentic workflow**: proactive ambiguity detection, semantic intent and entity routing, dynamic schema relationship mapping, multi-pass AST safety validation, an automated self-healing correction loop with database execution feedback, and automated narrative synthesis with charting recommendations.

Key System Objective: Empower decision-makers with zero-latency business insights while strictly guaranteeing enterprise security, AST-level read-only isolation, credential masking, and zero database mutations.

2. End-to-End Architectural Pipeline

The application operates via an integrated 7-stage deterministic and LLM-assisted execution pipeline:

- Step 1: Ambiguity & Policy Check:** Validates user query against restricted security keywords (passwords, master keys) and checks for underspecified business dimensions (prompts user with multiple-choice clarifying options).
- Step 2: NLP Intent & Entity Parser:** Decomposes the prompt into structured components: analytical intent (aggregation, listing, drill-down), entities (retailers, distributors, SKUs), temporal ranges, and condition filters.
- Step 3: Knowledge Graph Resolution:** Traverses schema relationship metadata to identify required tables and joins (foreign key paths) without overwhelming the LLM with unnecessary schema tables.
- Step 4: Prompt Construction & Generation:** Injects the selected schema subset, dynamic context, and few-shot examples into Ollama (Qwen2.5-Coder:7b) to generate clean, optimized SQL.
- Step 5: AST & Semantic Safety Validation:** Parses the generated SQL Abstract Syntax Tree (AST) to ensure strict SELECT-only operations, valid table/column permissions, and absence of SQL injection vectors.
- Step 6: EXPLAIN Plan & Self-Correction Loop:** Evaluates database execution cost via EXPLAIN. If syntax, AST, or execution errors occur, the error is fed back into the prompt for automated self-healing (up to 3 retries).
- Step 7: Execution & Executive Synthesis:** Executes the read-only query against MySQL/database engine, calculates confidence scores, and formats the output into natural language takeaways, interactive DataGrids, and recommended charts.

User Input	FastAPI Gateway	Agentic Core	Validation & DB	Synthesis & UI
Natural Language Query <i>'Top 5 retailers by wallet balance in July'</i>	REST Endpoint /api/query Session & Privacy State	Intent Extraction Relationship Resolver Ollama Qwen2.5-Coder	AST Validator EXPLAIN Cost MySQL Read-Only DB	Executive Narrative Interactive DataGrid Recharts / Bar / Pie

3. Core Subsystems & Repository Structure

Subsystem	Key Files / Modules	Technical Capabilities & Responsibility
Agent Orchestration	app/agent/sql_agent.py app/agent/ambiguity_checker.py app/agent/nlp_understanding.py	Coordinates multi-step reasoning, intent parsing, clarifying question prompts, error retries, and overall query lifecycle.
Knowledge & Schema	app/knowledge/knowledge_graph.py app/knowledge/relationship_resolver.py app/database/metadata_extractor.py	Maintains schema metadata, relationship mappings, foreign key graphs, and dynamic context injection to prevent hallucinations.
Validation & Safety	app/validator/sql_ast_validator.py app/validator/semantic_sql_validator.py app/database/read_executor.py	Enforces read-only safety, AST token inspection, SQL injection blocking, EXPLAIN query cost checks, and table access whitelisting.
LLM & Inference	app/llm/sql_generator.py app/prompt/prompt_builder.py cloud_gpu_setup.md	Interface to Ollama (qwen2.5-coder:7b). Supports local host and remote Cloud GPU endpoints via Ngrok / LocalTunnel tunnels.
Narrative & Synthesis	app/agent/response_synthesizer.py app/agent/storyteller.py app/utils/summary_generator.py	Generates plain-language executive summaries, KPI extraction, insights, and recommended visualization types (bar, line, metric).
Modern Web Frontend	frontend/src/App.jsx CenterChat.jsx, DataGrid.jsx AutoChart.jsx, ArchitectureView.jsx	React + Vite application with live thinking step progress loader, paginated DataGrid with CSV/Excel/PDF export, dynamic charts, and ERD viewer.
Security & Privacy	app/database/config.py app/utils/privacy_manager.py secure_credentials.py	AES-256 Fernet credential encryption with master keys, SSL wire encryption, incognito modes, and schema drift monitoring.

4. Enterprise Security, Governance & Verification

To ensure production reliability in enterprise environments, the platform implements defense-in-depth safety controls:

- **Restricted Credential Firewall:** Automatically scans user input and blocks any queries targeting sensitive authentication data (e.g. passwords, password hashes, encryption keys, master keys, tokens).
- **Strict Read-Only Enforcement:** The database execution engine parses commands before execution, rejecting any mutation statements (INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE) with an immediate security violation.
- **AES-256 Fernet DB Encryption:** Database connection strings and credentials are encrypted on disk via Fernet encryption and decrypted only into volatile RAM using the master secret key.
- **Schema Drift Monitoring:** On server startup and periodic intervals, a schema drift detector compares live database tables and columns against the baseline catalog, flagging discrepancies before queries fail.
- **Automated Self-Healing Loop:** If the generated SQL encounters a syntax error or semantic join failure, the error stack trace is returned to the agent, prompting the LLM to self-correct up to 3 attempts.

5. Technology Stack

Category	Technologies & Frameworks	Role in System
Backend API	Python 3.10+, FastAPI, Uvicorn	High-concurrency async REST API and static server
LLM & AI	Ollama, Qwen2.5-Coder:7b	Locally or remotely hosted code LLM for SQL generation
Database	MySQL, SQLAlchemy, PyMySQL	Enterprise relational data store with read-only connection pooling
Frontend	React 18, Vite, Recharts, Lucide	Interactive single-page application with dark/light themes
Reporting & Export	ReportLab, Pandas, OpenPyXL	Document generation for CSV, Excel workbooks, and PDF reports
Security & Cryptography	Cryptography (Fernet), PyJWT	Payload encryption, token authorization, and secret storage

6. Operational Quick Start

To run the complete platform locally, execute the following commands:

Step	Command	Description
1. Pull LLM	ollama run qwen2.5-coder:7b	Downloads and starts the Ollama model instance.
2. Start Backend	python start_server.py (or uvicorn app.api.main:app --port 8000)	Initializes schema metadata, knowledge graph, and API on http://localhost:8000.
3. Launch Frontend	cd frontend && npm run dev	Starts Vite development server on http://localhost:3000.
4. Diagnostic Health	GET /api/llm/health	Validates LLM endpoint connectivity, model availability, and response latency.

Document Generated: September 02, 2026 | **Project:** Agentic-Analyst (sneha-nayak546/Agentic-Analyst)
This document is an executive technical overview reflecting the live architecture, agent pipelines, and deployment specifications.